

ЛР8

Тестовые сценарии с описанием и выводами:

Добавление тестовых данных:

```
BEGIN;

DELETE FROM work_schedule
WHERE employee_id IN (
    SELECT employee_id
    FROM employee
    WHERE login LIKE 'ACID_TEST_%'
);

DELETE FROM activity_log
WHERE employee_id IN (
    SELECT employee_id
    FROM employee
    WHERE login LIKE 'ACID_TEST_%'
);

DELETE FROM employee
WHERE login LIKE 'ACID_TEST_%';

DELETE FROM bakery_ingredient_stock
WHERE bakery_id IN (
    SELECT bakery_id
    FROM bakery
    WHERE name = 'ACID_TEST_BAKERY'
)
OR ingredient_id IN (
    SELECT ingredient_id
    FROM ingredient
    WHERE name LIKE 'ACID_TEST_%'
);

DELETE FROM ingredient
WHERE name LIKE 'ACID_TEST_%';

DELETE FROM bakery
WHERE name = 'ACID_TEST_BAKERY';

INSERT INTO bakery (name, address, open_time, close_time, rent_cost, phone)
VALUES (
    'ACID_TEST_BAKERY',
    'Тестовый адрес для сценариев ACID',
    '08:00',
    '22:00',
```

```

0,
'+7-000-000-00-00'
);

INSERT INTO ingredient (
    name,
    description,
    cost_per_kg,
    cost_per_piece,
    calories_per_100g,
    calories_per_piece,
    shelf_life_days,
    can_be_weight,
    can_be_piece
)
VALUES
(
    'ACID_TEST_STOCK_MAIN',
    'Основной ингредиент для Atomicity, Durability, non-repeatable read и
SAVEPOINT',
    100,
    NULL,
    300,
    NULL,
    30,
    TRUE,
    FALSE
),
(
    'ACID_TEST_PHANTOM_EXISTING',
    'Уже существующий ингредиент для phantom read',
    100,
    NULL,
    300,
    NULL,
    30,
    TRUE,
    FALSE
),
(
    'ACID_TEST_PHANTOM_NEW',
    'Ингредиент, запас которого будет добавлен во второй сессии',
    100,
    NULL,
    300,
    NULL,
    30,
    TRUE,
    FALSE
);

```

```

-- 4. Начальные остатки ингредиентов.
-- ACID_TEST_STOCK_MAIN: 1000 г
-- ACID_TEST_PHANTOM_EXISTING: 200 г
INSERT INTO bakery_ingredient_stock (
    bakery_id,
    ingredient_id,
    stock_qty_g,
    stock_qty_pcs,
    avg_daily_consumption_g,
    avg_daily_consumption_pcs
)
SELECT
    b.bakery_id,
    i.ingredient_id,
    CASE
        WHEN i.name = 'ACID_TEST_STOCK_MAIN' THEN 1000
        WHEN i.name = 'ACID_TEST_PHANTOM_EXISTING' THEN 200
    END AS stock_qty_g,
    0,
    0,
    0
FROM bakery b
JOIN ingredient i
    ON i.name IN ('ACID_TEST_STOCK_MAIN', 'ACID_TEST_PHANTOM_EXISTING')
WHERE b.name = 'ACID_TEST_BAKERY';

-- 5. Два активных кассира для write skew
-- Бизнес-правило
-- в тестовой кондитерской должен оставаться хотя бы один активный кассир
-- В схеме это правило не закреплено ограничением, поэтому возможен write skew
INSERT INTO employee (
    bakery_id,
    full_name,
    passport_no,
    role_code,
    hourly_rate,
    fixed_monthly_salary,
    courier_bonus_pct,
    min_hours_per_month,
    login,
    password_hash,
    is_active,
    hire_date
)
SELECT
    b.bakery_id,
    'ACID Test Cashier 1',
    'ACID-TEST-PASSPORT-1',
    'CASHIER',
    500,
    0,
    0,
    0,
    0,
    0,
    1,
    '2023-01-01'

```

```

0,
0,
'ACID_TEST_cashier_1',
'hash',
TRUE,
CURRENT_DATE
FROM bakery b
WHERE b.name = 'ACID_TEST_BAKERY'
UNION ALL
SELECT
    b.bakery_id,
    'ACID Test Cashier 2',
    'ACID-TEST-PASSPORT-2',
    'CASHIER',
    500,
    0,
    0,
    0,
    'ACID_TEST_cashier_2',
    'hash',
    TRUE,
    CURRENT_DATE
FROM bakery b
WHERE b.name = 'ACID_TEST_BAKERY';

COMMIT;

SELECT b.bakery_id, b.name
FROM bakery b
WHERE b.name = 'ACID_TEST_BAKERY';

SELECT i.name, bis.stock_qty_g
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
ORDER BY i.name;

SELECT login, full_name, role_code, is_active
FROM employee
WHERE login LIKE 'ACID_TEST_%'
ORDER BY login;

```

Сценарии с пояснениями и выводами:

```
/*
атомарность
Транзакция выполняется как единое целое: либо фиксируются все её успешные
действия
либо не фиксируется ничего
*/

-- RESET перед сценарием.
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 1000
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
  AND bis.ingredient_id = i.ingredient_id
  AND b.name = 'ACID_TEST_BAKERY'
  AND i.name = 'ACID_TEST_STOCK_MAIN';

DELETE FROM activity_log
WHERE info LIKE 'ACID_TEST_ATOMICITY%';

BEGIN;

-- Успешное изменение остатка внутри транзакции
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = stock_qty_g - 100
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
  AND bis.ingredient_id = i.ingredient_id
  AND b.name = 'ACID_TEST_BAKERY'
  AND i.name = 'ACID_TEST_STOCK_MAIN';

-- Ещё одно успешное действие внутри той же транзакции.
INSERT INTO activity_log (employee_id, action_type, info)
SELECT e.employee_id, 'ACID_TEST', 'ACID_TEST_ATOMICITY: списали 100 г'
FROM employee e
WHERE e.login = 'ACID_TEST_cashier_1';

-- ERROR: new row for relation "bakery_ingredient_stock" violates check
constraint ...
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = -1
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
  AND bis.ingredient_id = i.ingredient_id
  AND b.name = 'ACID_TEST_BAKERY'
  AND i.name = 'ACID_TEST_STOCK_MAIN';

-- неуспешная транзакция, откатываемся
ROLLBACK;
```

```
-- остаток снова будет 1000, а записей ACID_TEST_ATOMICITY нет
SELECT bis.stock_qty_g AS stock_after_rollback
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

SELECT COUNT(*) AS log_rows_after_rollback
FROM activity_log
WHERE info LIKE 'ACID_TEST_ATOMICITY%';

/*
Атомарность означает, что частичный результат транзакции не должен попасть в базу
Даже если первые команды были успешны, после ROLLBACK база вернулась к состоянию
до BEGIN
*/
```

```
/*
согласованность
Транзакция переводит базу из одного корректного состояния в другое корректное
состояние
Корректность поддерживается ограничениями схемы: CHECK, FOREIGN KEY, UNIQUE, NOT
NULL и т.д.
*/

UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 1000
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

BEGIN;

-- ошибка: нарушается CHECK (stock_qty_g >= 0)
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = -500
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

-- делаем откат после ошибки
ROLLBACK;

-- проверка: отрицательное значение не попало в таблицу
SELECT bis.stock_qty_g AS stock_after_check_violation
```

```

FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

BEGIN;

-- ошибка: bakery_id = -999999 не существует в таблице bakery
INSERT INTO bakery_ingredient_stock (
    bakery_id,
    ingredient_id,
    stock_qty_g,
    stock_qty_pcs,
    avg_daily_consumption_g,
    avg_daily_consumption_pcs
)
SELECT
    -999999,
    i.ingredient_id,
    100,
    0,
    0,
    0
FROM ingredient i
WHERE i.name = 'ACID_TEST_STOCK_MAIN';

ROLLBACK;

-- проверка: строки с несуществующей кондитерской нет
SELECT COUNT(*) AS invalid_fk_rows
FROM bakery_ingredient_stock
WHERE bakery_id = -999999;

/*
Согласованность обеспечивается не самой транзакцией, а правилами схемы
Если команда нарушает CHECK или FOREIGN KEY, PostgreSQL не даёт базе перейти
в некорректное состояние
*/

```

```

/*
изолированность
Параллельные транзакции не должны неконтролируемо мешать друг другу
Степень изолированности зависит от выбранного уровня изоляции
READ COMMITTED, REPEATABLE READ, SERIALIZABLE

нужны две SQL-сессии
*/

```

```

UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 1000
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

--- READ COMMITTED

-- сессия 1
BEGIN ISOLATION LEVEL READ COMMITTED;
SELECT current_setting('transaction_isolation') AS isolation_level;
SELECT bis.stock_qty_g AS session_1_first_read
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

-- сессия 2
BEGIN;
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 777
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
COMMIT;

-- сессия 1
-- второй SELECT увидит 777
SELECT bis.stock_qty_g AS session_1_second_read
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
ROLLBACK;

--- REPEATABLE READ

-- RESET
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 1000
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

```



```

-- сессия 1
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT current_setting('transaction_isolation') AS isolation_level;
SELECT bis.stock_qty_g AS session_1_first_read
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';

-- сессия 2
BEGIN;
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 777
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
COMMIT;

-- сессия 1
-- второй SELECT всё ещё увидит 1000
SELECT bis.stock_qty_g AS session_1_second_read
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
ROLLBACK;

/*
Изолированность не означает, что транзакции вообще не видят друг друга
Она означает, что СУБД задаёт правила видимости параллельных изменений
На READ COMMITTED новый запрос внутри той же транзакции может увидеть новый
committed-результат
На REPEATABLE READ транзакция работает с одним снимком данных
*/

```

```

/*
долговечность
Если транзакция успешно завершилась COMMIT, результат должен сохраниться в базе
Он не должен исчезнуть после закрытия SQL-окна, переподключения к БД или запуска
нового SELECT

Выполняем COMMIT
Закрываем текущую сессию или открываем новую
Видим зафиксированное значение

```

Запускается в одной сессии + затем проверяется в новой сессии

*/

-- RESET

```
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 1000
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
```

-- сессия 1 - фиксируем новое значение

BEGIN;

```
UPDATE bakery_ingredient_stock bis
SET stock_qty_g = 1234
FROM bakery b, ingredient i
WHERE bis.bakery_id = b.bakery_id
      AND bis.ingredient_id = i.ingredient_id
      AND b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
COMMIT;
```

-- сессия 1 - проверка сразу после COMMIT

```
SELECT bis.stock_qty_g AS value_after_commit
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
```

-- сессия 2

-- выполнение заново уже после переподключения

-- осталось 1234

```
SELECT bis.stock_qty_g AS value_after_reconnect
FROM bakery_ingredient_stock bis
JOIN bakery b ON b.bakery_id = bis.bakery_id
JOIN ingredient i ON i.ingredient_id = bis.ingredient_id
WHERE b.name = 'ACID_TEST_BAKERY'
      AND i.name = 'ACID_TEST_STOCK_MAIN';
```

/*

До COMMIT изменение является временным и может быть отменено ROLLBACK
После COMMIT изменение становится частью состояния базы и видимо новым
подключениям

*/